protobuf配置和使用

Posted on 2023-05-15 16:26:48 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2Pp1SSXN9MDHMFG9WtkayoC3BeC)

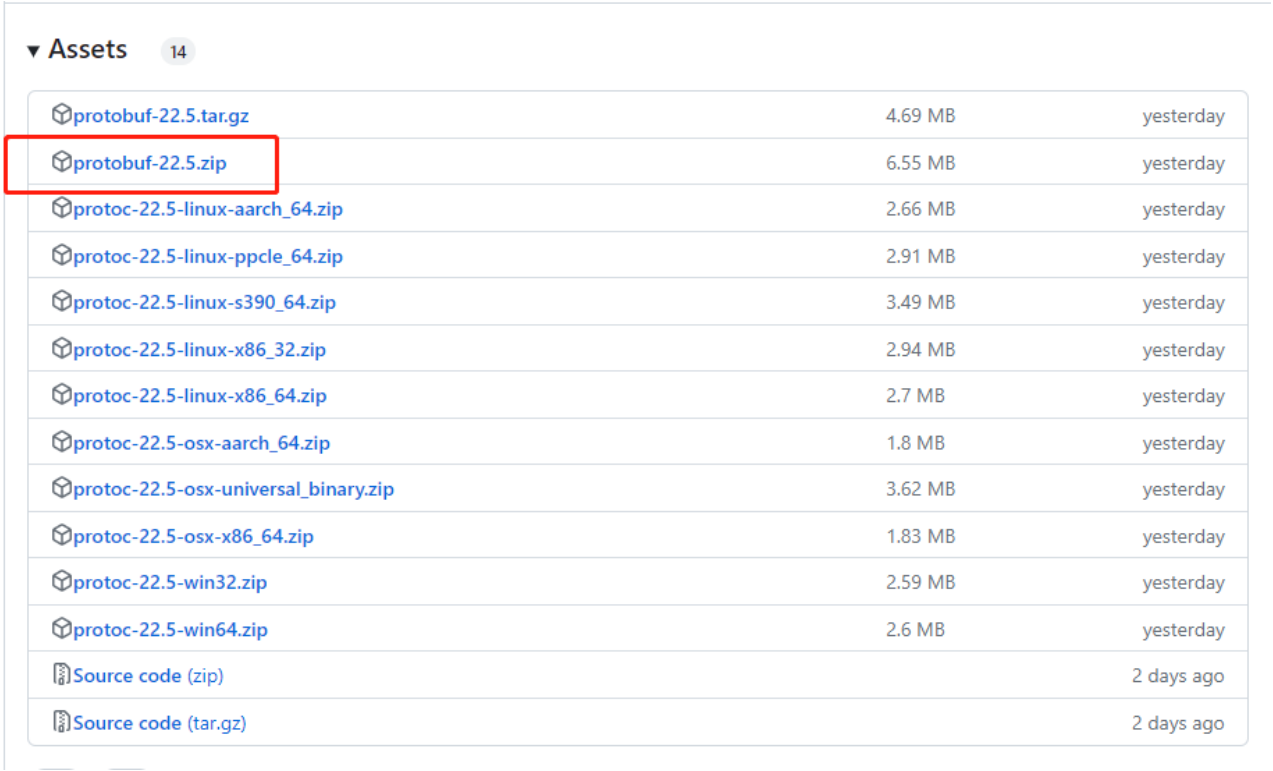
protobuf简介

Protocol Buffers（简称 Protobuf）是一种轻便高效的序列化数据结构的协议，由 Google 开发。它可以用于将结构化数据序列化到二进制格式，并广泛用于数据存储、通信协议、配置文件等领域。
我们的逻辑是有类等抽象数据构成的，而tcp是面向字节流的，我们需要将类结构序列化为字符串来传输。

下载protobuf安装包

官方下载地址
https://github.com/protocolbuffers/protobuf/releases (https://github.com/protocolbuffers/protobuf/releases)

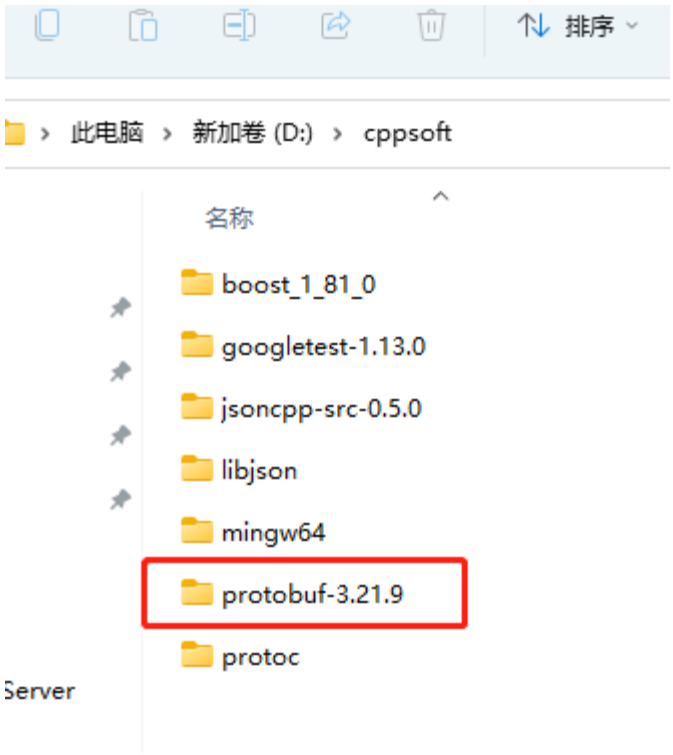
选择zip文件下载即可



因为官网下载速度较慢，我把下载好的protobuf存在网盘里了 https://pan.baidu.com/s/1Yg9Usdc3T-CYhyr9GiePCw?pwd=ng6x (https://pan.baidu.com/s/1Yg9Usdc3T-CYhyr9GiePCw?pwd=ng6x)

验证码ng6x

下载完后解压即可，我放在D盘cppsoft文件夹下



下载cmake

CMake是一个跨平台的开源构建工具，可以用于自动化构建、测试和打包C++代码。与其他构建工具相比，CMake的优点之一是它的配置文件具有可读性和可维护性，而且支持多种编译器、操作系统和构建工具。

使用CMake，您可以将C++项目的配置和生成分离开来，从而使其更加可重复，并且可以在不同的平台上进行构建和测试。CMake使用类似于脚本语言的语法来描述构建过程，并生成适用于各种构建系统的构建文件（例如Makefile、Visual Studio解决方案等）。

以下是CMake的一些功能：

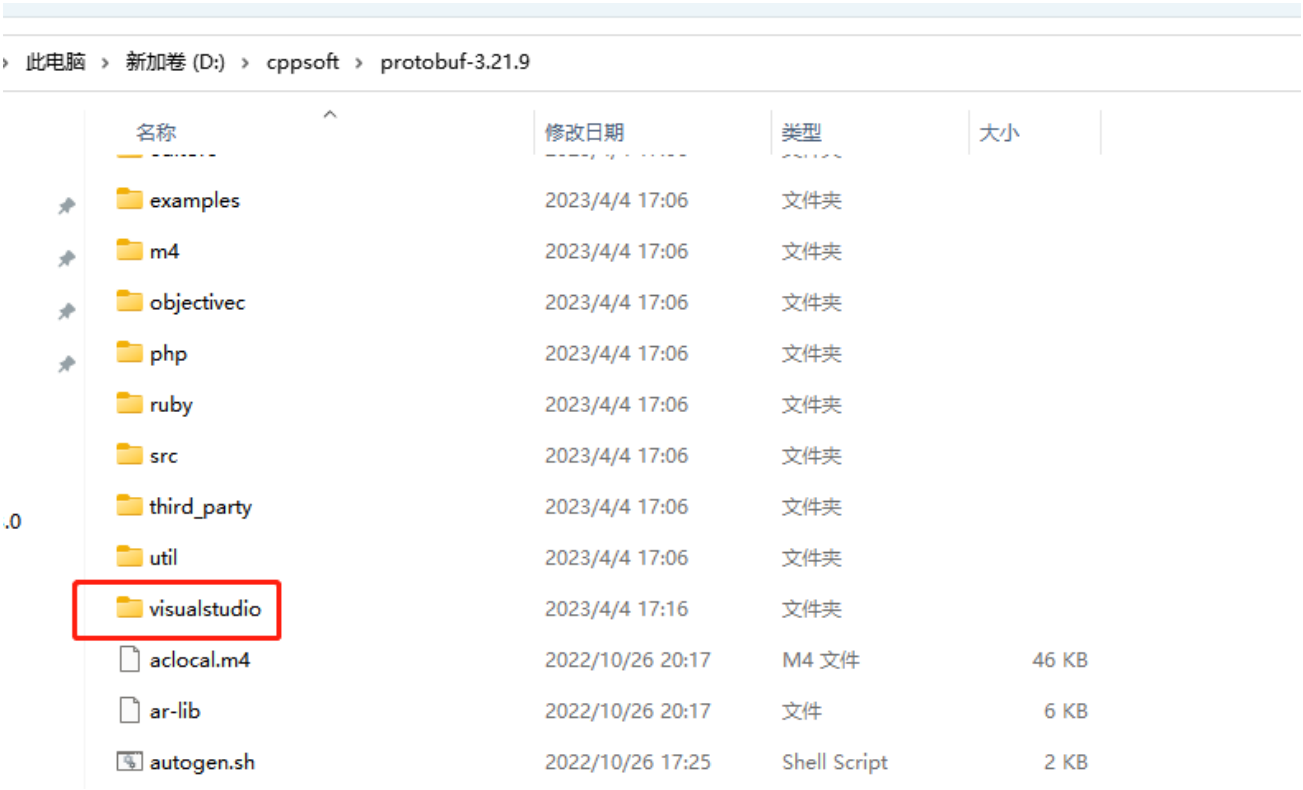
- 自动生成包含目录、库和源文件列表的Makefile
- 支持多种编译器和操作系统
- 可以自动生成安装程序和打包文件
- 跨平台支持
- CMakeLists.txt 配置文件编写简单，易于维护

总之，CMake是一个强大的构建工具，可以大大简化C++项目的构建过程，并提高可移植性和可重复性。如果您正在寻找一个跨平台的自动构建工具来构建您的C++代码，请考虑使用CMake。

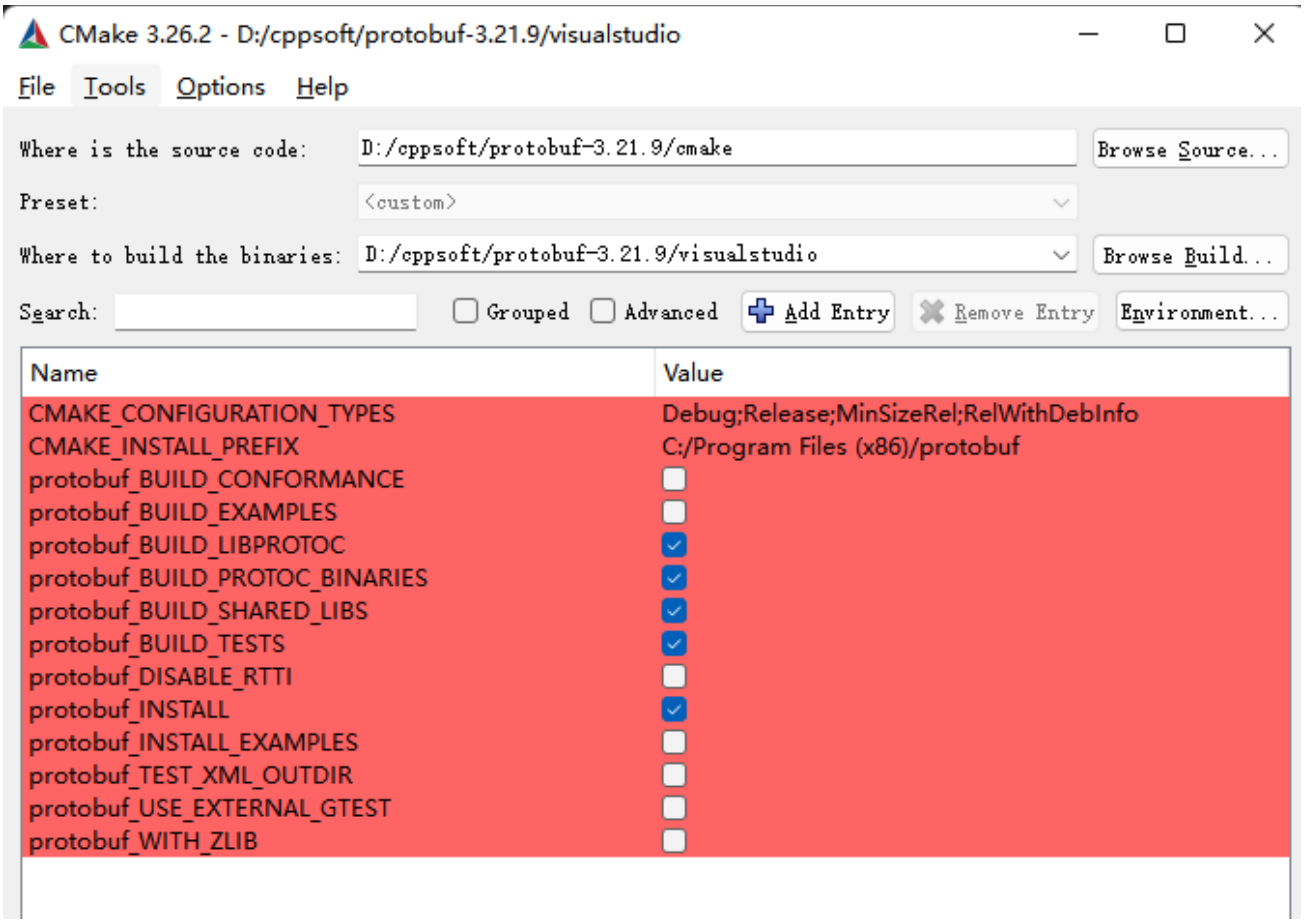
我们要在windows上生成protobuf对应的库，就需要用到cmake。
官方下载地址https://cmake.org/download/ (https://cmake.org/download/)， 选择Windows Source下载。
由于官方下载较慢，可以去我的网盘下载
https://pan.baidu.com/s/1Yg9Usdc3T-CYhyr9GiePCw?pwd=ng6x (https://pan.baidu.com/s/1Yg9Usdc3T-CYhyr9GiePCw?pwd=ng6x)

编译protobuf

在protobuf中创建一个文件夹，名字叫做visualstudio，你可以叫其他的名字,如下图

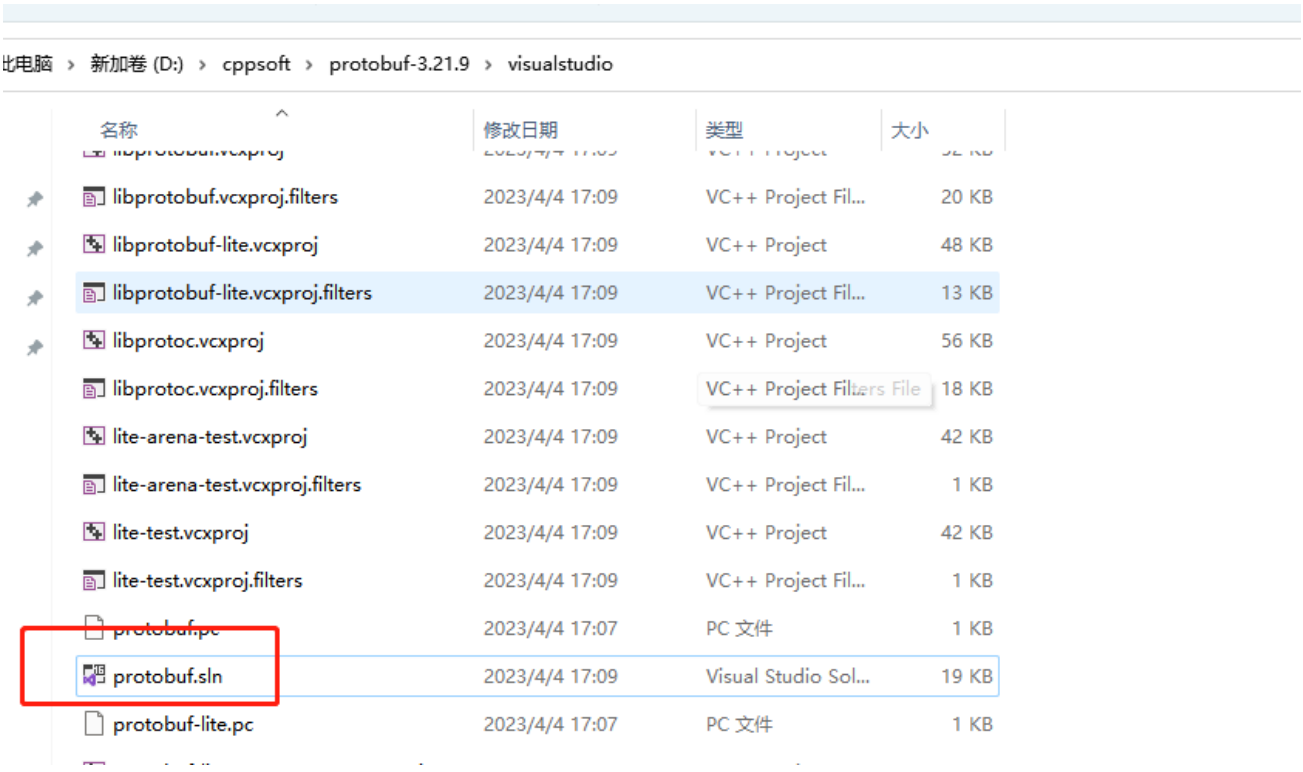


然后我们打开刚才下载的cmake桌面程序,点击后
source code 选择protobuf源码路径下的cmake目录
build binaries目录选择我们刚刚创建的visualstudio目录
生成选项可以依据自己要求生成。

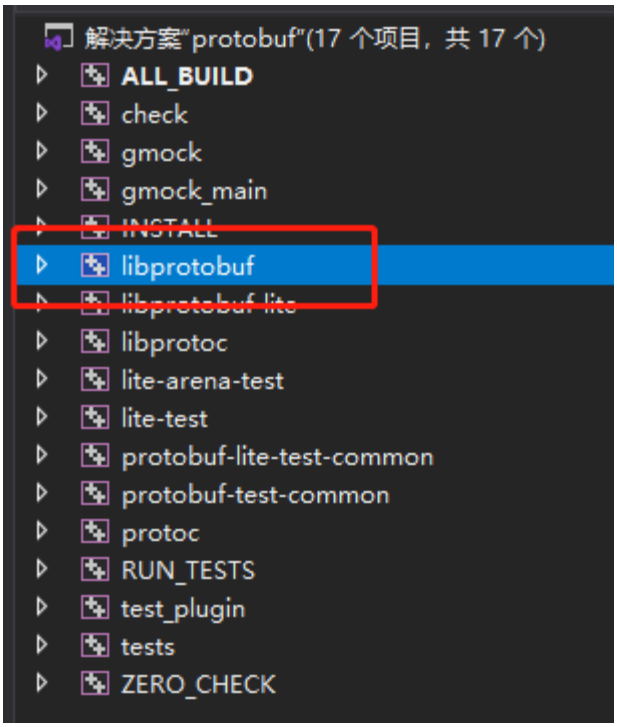


点击config，再点击generate就可以生成了。

生成后我们可以看到protobuf的visualstudio文件夹内生成了.sln文件。



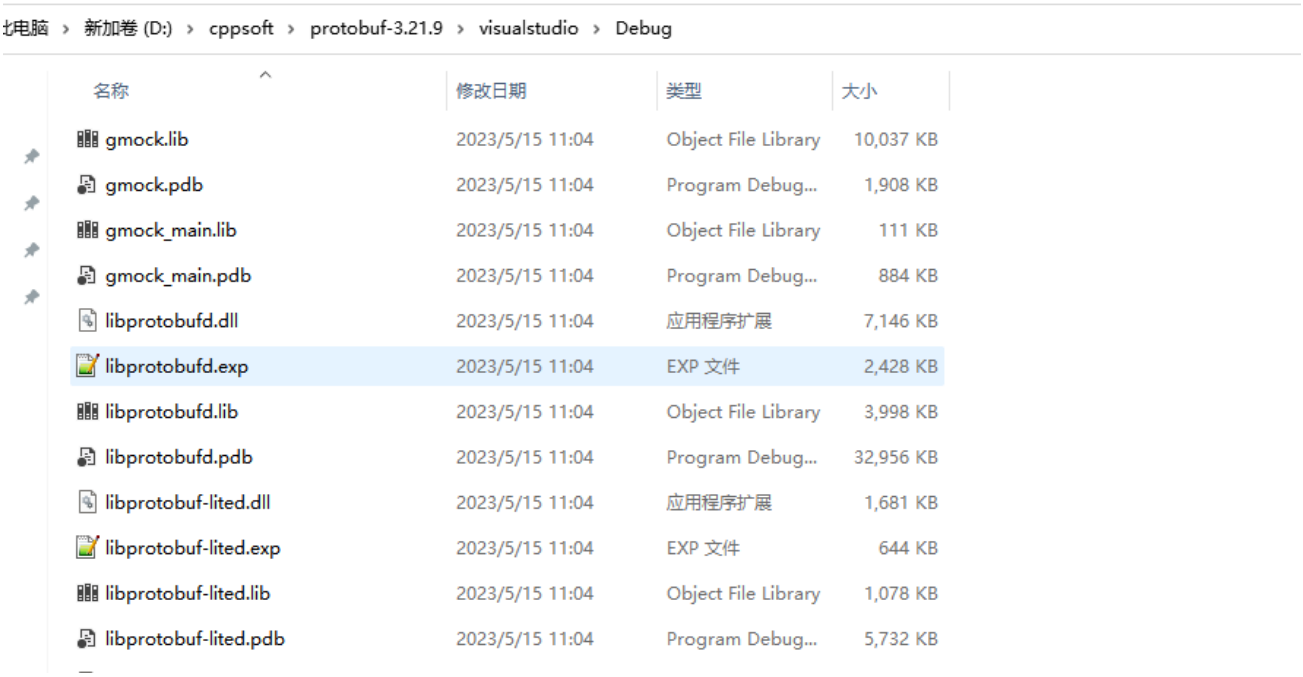
双击.sln文件，我的是默认用visual studio2019打开



可以单独选择libprotobuf编译，也可以选择All Build编译

编译时可以选择debug和release，建议每一个都编译一遍，这样就有debug和release两个版本的库了。

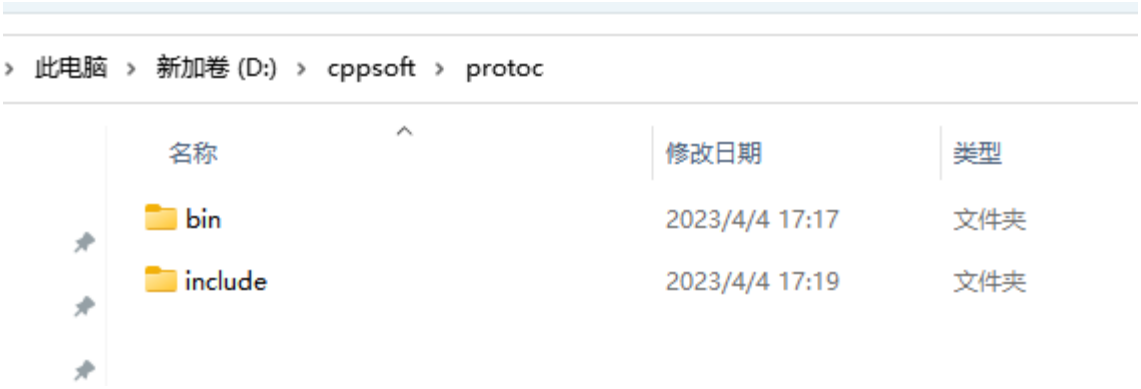
我选择的是X64位的debug模式编译，生成的库在visualstudio目录下的Debug文件夹里。



可以看到我们生成了protobuf相关的lib库和dll库，一个是静态类型的，一个是动态类型的。

接下来我创建一个新的文件夹D:\cppsoft\protoc

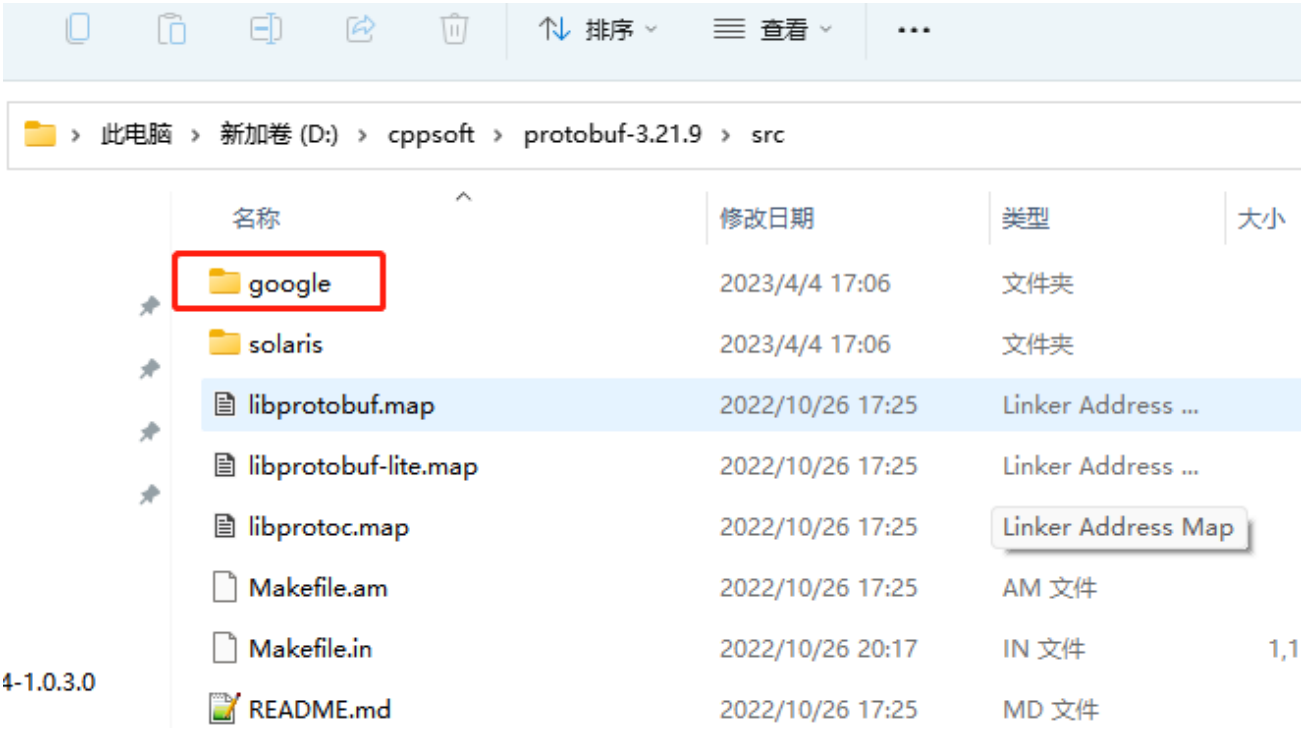
然后在该文件夹内创建一个bin文件夹(用来存储刚才protobuf生成的库)和include文件夹(用来存储protobuf的头文件)



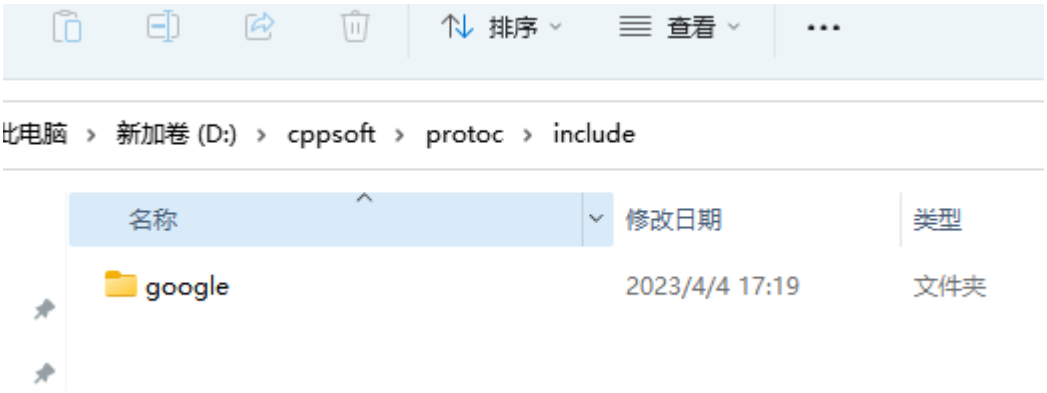
将libprotobufd.lib和libprotocd.lib， 以及protoc.exe拷贝到bin目录下，当然为了偷懒，可以将刚才Debug目录下的所有内容都拷贝到bin目录也可以。

将protobuf文件夹下src文件夹里的google文件夹及其内容拷贝到protoc的include文件夹

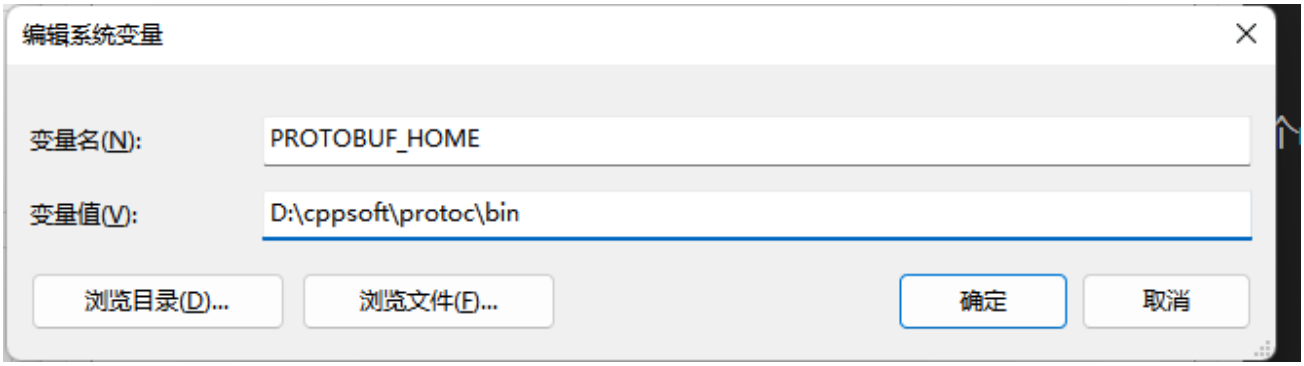
将编译目录的google文件夹



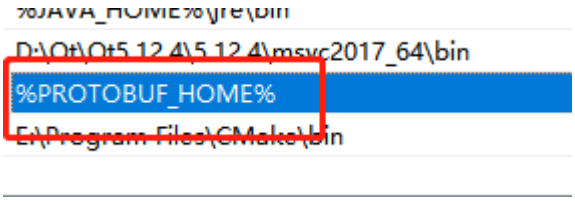
拷贝到protoc的include文件夹



到此为止，我们protobuf的库生成工作就完成了。
因为我们要用到protoc命令，所以要将该命令配置到环境变量，在系统环境变量里添加一个环境变量PROTOBUF_HOME,
设置它的值为D:\cppsoft\protoc\bin



然后将该值添加到系统的path路径即可,格式为%PROTOBUF_HOME%



这样我们就可以直接使用protoc.exe了。

visual stuido配置protobuf

我们新建一个控制台项目，在项目属性中，配置选择Debug，平台选择X64，选择VC++目录，
在包含目录中添加 D:\cppsoft\protoc\include
在库目录中添加 D:\cppsoft\protoc\bin

在链接器的输入选项中添加protobuf用到的lib库

1. libprotobufd.lib
2. libprotocd.lib

到此，visual studio 的protobuf配置完毕。

生成pb文件

要想使用protobuf的序列化功能，需要生成pb文件，pb文件包含了我们要序列化的类信息。

我们先创建一个msg.proto，该文件用来定义我们要发送的类信息

```
1. syntax = "proto3";
2. message Book
3. {
4.    string name = 1;
5.    int32 pages = 2;
6.    float price = 3;
7. }
```

这个文件定义了一个名为Book的消息类型，包含三个字段：name、pages和price。其中每个字段都有一个数字标识符，用于标识该字段在二进制流中的位置。
我们使用protoc.exe 基于msg.proto生成我们要用的C++类
在proto所在文件夹执行如下命令

```
1. protoc --cpp_out=. ./msg.proto
```

—cpp_out= 表示指定要生成的pb文件所在的位置
./msg.proto 表示msg.proto所在的位置，因为我们是在msg.proto所在文件夹中执行的protoc命令,所以是当前路径即可。
执行后，会看到当前目录生成了msg.pb.h和msg.pb.cc两个文件，这两个文件就是我们要用到的头文件和cpp文件。
我们将这两个文件添加到项目里，然后在主函数中包含msg.pb.h，做如下测试

```
#include <iostream>
#include "msg.pb.h"
int main()
{
    Book book;
    book.set_name("CPP programing");
    book.set_pages(100);
    book.set_price(200);
    std::string bookstr;
    book.SerializeToString(&bookstr);
    std::cout << "serialize str is " << bookstr << std::endl;
    Book book2;
    book2.ParseFromString(bookstr);
    std::cout << "book2 name is " << book2.name() << " price is "
        << book2.price() << " pages is " << book2.pages() << std::endl;
    getchar();
}
```

上面的demo中将book对象先序列化为字符串，再将字符串反序列化为book2对象。
这样就是visual studio 配置和使用protobuf的方法。

在网络中的应用

先为服务器定义一个用来通信的proto

```
syntax = "proto3";
message MsgData
{
   int32  id = 1;
   string data = 2;
}
```

id代表消息id，data代表消息内容
我们用protoc生成对应的pb.h和pb.cc文件
将proto,pb.cc,pb.h三个文件复制到我们之前的服务器项目里并且配置。

我们修改服务器接收数据和发送数据的逻辑
当服务器收到数据后，完成切包处理后，将信息反序列化为具体要使用的结构,打印相关的信息，然后再发送给客户端

```
1.  MsgData msgdata;
2.  std::string receive_data;
3.  msgdata.ParseFromString(std::string(_recv_msg_node->_data, _recv_msg_node->_total_le
    n));
4.  std::cout << "recevie msg id  is " << msgdata.id() << " msg data is " << msgdata.data() << en
    dl;
5.  std::string return_str = "server has received msg, msg data is " + msgdata.data();
6.  MsgData msgreturn;
7.  msgreturn.set_id(msgdata.id());
8.  msgreturn.set_data(return_str);
9.  msgreturn.SerializeToString(&return_str);
10. Send(return_str);
```

同样的道理，客户端在发送的时候也利用protobuf进行消息的序列化，然后发给服务器

```
1.  MsgData msgdata;
2.  msgdata.set_id(1001);
3.  msgdata.set_data("hello world");
4.  std::string request;
5.  msgdata.SerializeToString(&request);
```

总结

本文介绍了visual studio如何配置并使用protobuf，以及在服务器通信中的使用
源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(4135) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(585) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)